

Youtube Downloader - Developer Documentation v0.2

This section provides information for developers who want to understand, modify, or contribute to the refactored YouTube Downloader application.

1. Project Structure (Refactored)

The application is now organized into the following packages and modules:

```
yt_downloader/  
├─ core/  
│   └─ downloader.py (VideoDownloader - Handles video downloading via yt_dlp)  
├─ ui/  
│   ├── **init**.py (Makes 'ui' a package)  
│   ├── about_window.py (AboutDialog - Displays application information)  
│   ├── download_thread.py (DownloadThread - Downloads videos in a separate thread)  
│   ├── main_window.py (Minimal - creates AppController and UIManager, starts event loop)  
│   ├── ui_manager.py (UIManager - Manages all UI elements and interactions)  
│   ├── search_controller.py (SearchController - Manages search logic and caching)  
│   ├── download_controller.py (DownloadController - Manages download logic)  
│   ├── settings_manager.py (SettingsManager - Manages application settings)  
│   └─ file_controller.py (FileController - Manages file operations)  
├─ helper/  
│   ├── **init**.py (Makes 'helper' a package)  
│   ├── search_worker.py (SearchWorker - Performs YouTube searches in a thread)  
│   └─ file_opener_thread.py (FileOpenerThread - Opens files in a separate thread)  
├─ docs/  
│   ├── developer_documentation_v0.2.md  
│   ├── yt_downloader_dev_documentation_v0.2.pdf  
│   └─ yt_downloader_user_guide.pdf  
├─ requirements.txt (Lists required Python packages)  
└─ main.py (Bootstraps the application, connects components)
```

2. Dependencies

- **PyQt6:** The GUI framework.
- **requests:** (Not directly used, but often useful for web tasks)
- **yt_dlp:** The core library for downloading videos.
- **qtawesome:** Provides Font Awesome icons.

3. Code Overview

3.1. `main.py`

- **Responsibility:** Application entry point. Bootstraps the application.
- **Functionality:**

- Creates instances of `SettingsManager`, `UIManager`, `SearchController`, `DownloadController`, `FileController` and `AppController`.
- Connects signals and slots between these objects to establish the application's workflow.
- Starts the Qt event loop.
- Creates a `QMainWindow` and sets a `UIManager` instance as central widget

3.2. `core/downloader.py` - `VideoDownloader`

- **Responsibility:** Handles the actual video downloading using `yt_dlp`.
- **Methods:**
 - `download(url, quality, save_path, progress_callback=None)` : Downloads a video.
 - `_progress_hook(progress_callback)` : Internal helper for progress updates.

3.3. `ui/about_window.py` - `AboutDialog`

- **Responsibility:** Displays "About" information (author, GitHub link, etc.).

3.4. `ui/download_thread.py` - `DownloadThread`

- **Responsibility:** Performs video downloads in a separate thread to prevent UI freezing.
- **Signals:**
 - `progress_signal(int)` : Emits download progress updates.
 - `finished_signal(str)` : Emits a message on successful completion.
 - `error_signal(str)` : Emits an error message on failure.

3.5. `helper/search_worker.py` - `SearchWorker`

- **Responsibility:** Performs YouTube searches in a separate thread.
- **Signals:**
 - `results_signal(list)` : Emits search results.
 - `error_signal(str)` : Emits an error message.

3.6. `helper/file_opener_thread.py` - `FileOpenerThread`

- **Responsibility:** Open file in a separate thread.
- **Signals:**
 - `finished` : Emits signal for finish opening.
 - `error` : Emits signal for error.

3.7. `ui/settings_manager.py` - `SettingsManager`

- **Responsibility:** Manages loading, saving, and accessing application settings.
- **Methods:**
 - `load_settings()` : Loads settings from a JSON file.
 - `save_settings()` : Saves settings to a JSON file.
 - `get_setting(key, default=None)` : Retrieves a setting value.
 - `set_setting(key, value)` : Sets a setting value.

3.8. ui/ui_manager.py - UIManager

- **Responsibility:** Manages *all* aspects of the user interface.
- **Methods:**
 - `setup_ui()` : Creates and arranges all UI elements (widgets, layouts).
 - `set_dark_mode(is_dark_mode)` : Applies the dark or light theme.
 - `display_results(results)` : Displays search results in the list.
 - `clear_results()` : Clears the results list.
 - `set_status_message(message)` : Sets the text of the status label.
 - `set_progress_value(value)` : Sets the value of the progress bar.
 - `set_save_path(path)` : Updates the displayed save path.
 - `populate_quality_combo(video_info)` : Populates the quality options.
 - `show_message(title, message)` : show message dialog
 - `show_error(title, message)` : show error dialog
 - `update_quality_options` : Updates enable/disable option quality.
 - Event handler methods (e.g., `on_search_clicked`, `on_download_clicked`): These methods emit signals corresponding to user actions. They do *not* contain the application logic itself.
- **Signals:**
 - `search_requested(str)` : Emitted when the user clicks "Search".
 - `download_requested(str, str, str)` : Emitted when the user clicks "Download".
 - `cancel_requested()` : Emitted when the user clicks "Cancel".
 - `path_browse_requested()` : Emitted when the user clicks "Browse...".
 - `theme_change_requested(bool)` : Emitted when the user toggles the dark mode checkbox.
 - `help_user_requested` : Emitted when the user request to view help user doc.
 - `help_dev_requested` : Emitted when the user request to view help dev doc.
 - `about_requested` : Emitted when the user request to view about dialog.
- **Important:** This class handles UI *events* and updates the UI *state*, but it does *not* contain application logic (searching, downloading, etc.). It interacts with the `AppController` via signals and slots.

3.9. ui/search_controller.py - SearchController

- **Responsibility:** Manages the YouTube search process.
- **Methods:**
 - `search(query)` : Starts a search using `SearchWorker`.
 - `handle_results(results)` : Handles results from `SearchWorker`.
 - `handle_error(error_message)` : Handles errors from `SearchWorker`.
- **Signals:**
 - `search_results_ready(list)` : Emits search results.
 - `search_error(str)` : Emits an error message.
- **Internal:** Uses a `search_cache` to store search results.

3.10. ui/download_controller.py - DownloadController

- **Responsibility:** Manages the video download process.
- **Methods:**

- `download(url, quality, save_path)` : Starts a download using `DownloadThread`.
- `cancel_download()` : Cancels the current download.
- **Signals:**
 - `download_progress(int)` : Emits download progress updates.
 - `download_finished(str)` : Emits a message on successful completion.
 - `download_error(str)` : Emits an error message.

3.11 `ui/file_controller.py` - `FileController`

- **Responsibility:** Manages file operations.
- **Methods:**
 - `open_file` : Opens a file specified by the file using a separate thread.
- **Signals:**
 - `file_opened` : Emits open file success.
 - `file_open_error` : Emits an error message.

3.12. `AppController` (in `main.py`)

- **Responsibility:** Coordinates the entire application. Acts as a mediator between the UI, search, download, file and settings components.
- **Methods:**
 - Slots connected to signals from `UIManager`, `SearchController`, `DownloadController`, and `FileController`. These slots contain the *application logic*.
 - `start()` : Initializes any necessary state.
 - `on_item_selected` : populate quality option when item select
 - `set_dark_mode` : set dark mode.
 - `browse_for_path` : browse path to save file.
 - `start_download` : start download
 - `show_about_dialog` : show about dialog.

4. Key Concepts (Refactored)

- **Separation of Concerns:** Each class has a single, well-defined responsibility.
- **Model-View-Controller (MVC) Variant:** The architecture loosely follows the MVC pattern:
 - **Model:** `SettingsManager`, `SearchWorker`, `DownloadThread`, and the data they manage.
 - **View:** `UIManager` (and `AboutDialog`).
 - **Controller:** `AppController`, `SearchController`, `DownloadController`, `FileController`.
- **Signal/Slot Mechanism:** Used extensively for communication between objects, especially between threads and the UI.
- **Threading:** `SearchWorker` and `DownloadThread` run in separate threads to prevent UI freezes.
- **Caching:** `SearchController` caches search results to improve performance.

5. Potential Improvements

- **Asynchronous `yt_dlp`:** Explore using `yt_dlp`'s asynchronous API (with `asyncio`) for potentially better performance.

- **More Detailed Error Handling:** Provide more specific error messages and handling for various `yt_dlp` exceptions.
- **Configuration File:** Allow users to customize more settings via a configuration file.
- **Playlist Downloading:** Add support for downloading entire YouTube playlists.
- **Download Resumption:** Implement the ability to resume interrupted downloads.
- **Testing:** Add unit tests and integration tests to ensure code quality and prevent regressions.

6. GitHub Repository

The source code for this project is available on GitHub: <https://github.com/LongKelvin/yt-downloader>

A pre-built, portable `.exe` version of the application can be found on the releases page:

<https://github.com/LongKelvin/yt-downloader/releases/tag/v0.1>

7. Building a Portable Executable (.exe)

(Instructions remain the same as before - using PyInstaller)

1. Install PyInstaller:

```
pip install pyinstaller
```

2. Build the Executable:

Open a terminal in the project directory (where `main.py` is located) and run:

```
pyinstaller --onefile --windowed --name "YouTube Downloader" main.py --collect-all
```

- `--onefile` : Creates a single executable file.
- `--windowed` : Prevents a console window from appearing when the `.exe` is run.
- `--name "YouTube Downloader"` : Sets the name of the executable file.
- `main.py` : Specifies the main Python script.
- `--collect-all qtawesome` : Collects all qtawesome files, this is a crucial step.

3. Find the Executable:

After the build process completes, the `.exe` file will be located in the `dist` folder (which PyInstaller creates).

4. Copy `user_guide.pdf`:

Copy the `user_guide.pdf` to the `dist` folder so it's in same directory with the executable file.

Author and Project Information

- **Author:** Long Kelvin
- **Project Name:** yt-downloader
- **GitHub Repository:** <https://github.com/LongKelvin/yt-downloader>
- **Release Date:** March 2025
- **Version:** 0.2
- **Contact:** longkelvin101099@gmail.com